

A Numerical Approach to Calculus

Implementation of Taylor Series with Java and Python

Jaehoon Song

March 28, 2026

Introduction

- **Programming fundamentals:** We start with how **abstraction** and the **operating system** stack (OS, shell, runtimes like the JVM and Python) hide hardware details and make development tractable.
- **Mathematical abstraction:** We then study the **Taylor series**—another abstraction: replacing a function with an (in principle infinite) polynomial built from derivatives and powers of x .
- **Irrational values in practice:** Machines do not store exact irrationals; they use **finite** floating-point words and **truncated** series so values are represented as well-chosen approximations, not infinite decimals.
- **Finite logic vs. infinite theory:** Computer programs run for finitely many steps at fixed precision, yet they can still embody “infinite” calculus ideas by stopping after enough terms and living inside tolerances and error bounds.

Abstraction

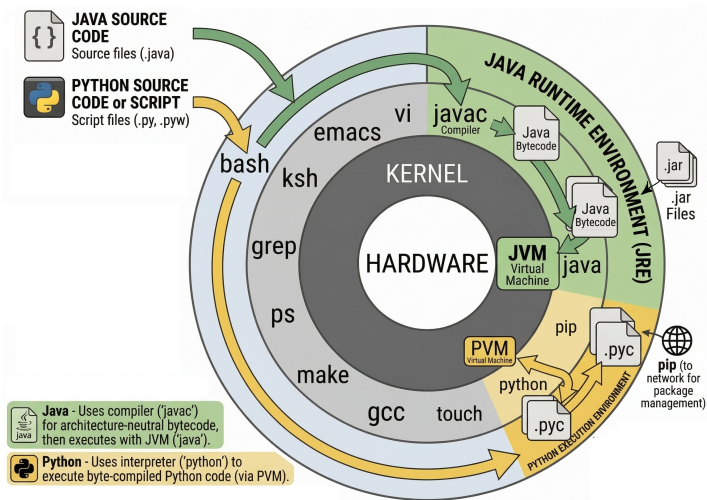
- **Abstraction:** Hiding complex details behind a simpler interface. (You don't need to know how it works, just what it does.)
- **OS Abstraction:** abstraction of hardware resource management and system calls.
- **Shell Abstraction:** command-line interface, abstraction of interacting with the OS and executing utilities.
- **Runtime Abstraction (JDK/Python):** abstraction of machines, running code on a virtual machine (JVM) or interpreter.

Script Automation: abstraction of environment setup; `dev.bat` / `dev.sh` will be provided to quickly set up your environment for these layers.

Next Step

Let's give a quick overview of how the OS, Shell, JDK, and Python relate by visualizing their place in the system hierarchy.

Operating System & Language Abstraction



Environment Setup & Detection

- **Finding the Tools:** When a command is typed (`javac Main.java`), the shell searches the system `PATH` environment variable.
- **PATH Variable:** A semicolon- or colon-separated list of directories containing executables.
- **Example Detection Sequence:**
 - ① User types command.
 - ② Shell looks in directories defined in `PATH`.
 - ③ First executable found is run (or error).
- **Automation via `dev.sh` / `dev.bat`:** Conceptually, these scripts modify the local session environment to automatically detect these tools.

Conceptual `dev.sh` Snippet

```
export JDK_HOME="/path/to/jdk-17"  
export PYTHON_HOME="/path/to/python3.9"  
export PATH="$JDK_HOME/bin:$PYTHON_HOME/bin:$PATH"
```

Abstraction: Example of Minecraft Modding



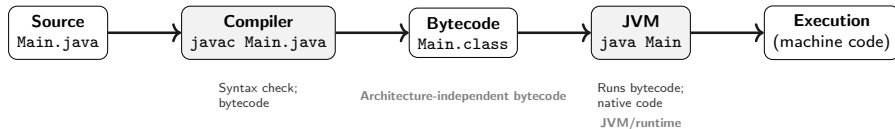
New blocks and items show up in the world long before you worry about rendering meshes or chunk storage—that is the **player-facing** side of abstraction.

Abstraction: Adding Content with Java & Game Libraries

```
4  import net.minecraft.block.Block;
5  import net.minecraft.item.BlockItem;
6  import net.minecraft.item.Item;
7  import net.minecraft.registry.Registries;
8  import net.minecraft.registry.Registry;
9  import net.minecraft.util.Identifier;
10
11  public class ModBlocks {
12      ⬮
13
14
15      private static Block registerBlock(String name, Block block) {
16          registerBlockItem(name, block);
17          return Registry.register(Registries.BLOCK, Identifier.of(TutorialMod.MOD_ID, name), block);
18      }
19
20      private static void registerBlockItem(String name, Block block) {
21          Registry.register(Registries.ITEM, Identifier.of(TutorialMod.MOD_ID, name),
22                          new BlockItem(block, new Item.Settings()));
23      }
24
25      public static void registerModBlocks() {
26          TutorialMod.LOGGER.info("Registering Mod Blocks for " + TutorialMod.MOD_ID);
27      }
28  }
29
```

Modding APIs (Registry, block/item types, identifiers) **abstract** the heavy lifting: the libraries are the abstraction layer for “adding” things.

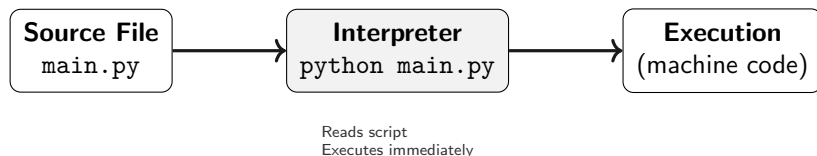
Java: Compile-Once, Run-Everywhere



Java Compilation Workflow

The Main.java is first compiled into a platform-independent format (bytecode) and then run by the JVM (Java Virtual Machine), **abstracting away system differences**.

Python: Interpreted & Dynamic



Python Execution Workflow

The `main.py` is interpreted and executed line-by-line, which speeds up the write-test loop compared to a compile-then-run Java workflow, **abstracting away system differences**.

Note: Typical pure-Python code runs slower than comparable JVM bytecode (and usually slower than native code).

Background: The Computational Problem

Hardware Limitations

Digital circuits are inherently limited to basic arithmetic operations:

- Addition and Subtraction
- Multiplication and Division

Continuous functions, such as $\sin(x)$, cannot be stored as an infinite lookup table. Instead, these values must be approximated using iterative algorithms.

Abstraction: The Taylor Series Expansion

The $\sin(x)$ function can be represented as an infinite sum of powers:

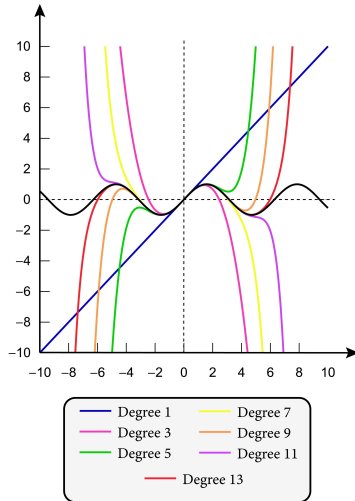
$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$$

This expansion is simplified into a sequence of polynomial terms:

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Higher accuracy is achieved as more terms are calculated.

Abstraction: Taylor Series of $\sin(x)$



This is what you guys have been taught or will be taught in high school calculus classes!

Implementation in Java

The mathematical summation is translated into a for loop.

Java Logic

```
double x = 1.0;
double result = 0;

for (int i = 0; i < 5; i++) {
    double term = Math.pow(x, 2*i + 1) / factorial(2*i + 1);
    if (i % 2 == 0) {
        result += term;
    } else {
        result -= term;
    }
}
```

Implementation in Python

The mathematical summation is translated into a for loop.

Python Logic

```
x = 1.0
result = 0
for i in range(5):
    term = x**(2*i + 1) / factorial(2*i + 1)
    if i % 2 == 0:
        result += term
    else:
        result -= term
```

Validation of Results

The approximation is compared against the standard library:

- **Taylor Approximation (5 terms):** 0.84147101...
- **Java Math.sin(1):** 0.84147098...
- **Python math.sin(1):** 0.84147098...

Conclusion

Mathematical abstraction is bridged by software loops. This logic serves as the foundation for modern computational tools.

Why This Math Matters: Trig, exp, and Linear Algebra

- **Games and engines (e.g. Unity):** Rotations, camera/view transforms, and animation use trig everywhere; lighting and motion often combine trig with linear algebra (vectors, matrices).
- **Physics simulation:** Springs, waves, and oscillations use sin and cos; damping, growth, and half-lives use exp (exponential decay and continuous-time models).
- **AI and machine learning:** Networks are built from matrix multiplies and linear algebra; softmax and many probabilistic models use exp; classical ML (e.g. **linear regression**) is explicit linear algebra on data matrices.
- **Shared theme:** Transcendental functions and linear systems are how we turn continuous models into code that runs at finite precision—the same bridge as Taylor series for $\sin(x)$.